

Backend Engineering Assignment

Python API, Data Modeling & Async Processing

Objective

Build a backend service that manages customer support tickets and asynchronous ticket processing.

The exercise is designed to evaluate your ability to:

1. Design clean and maintainable Python backend services.
 2. Build well-structured REST APIs.
 3. Model relational data effectively using PostgreSQL.
 4. Apply backend engineering best practices (validation, schemas, enums, architecture, error handling, typing, etc.).
 5. Implement asynchronous/background processing using queues.
 6. Containerize and document a production-style development setup.
-

Technical Requirements

Recommended Stack

- Python 3.12+
- Flask or FastAPI
- PostgreSQL
- Docker & Docker Compose

Optional:

- AWS SQS or AWS services as a bonus
-

Functional Requirements

Domain

Build a small “Support Ticket Management System”.

The system should allow:

- Customers to create support tickets
 - Agents to update ticket statuses
 - Background workers to process tickets asynchronously
-

Core Features

1. Ticket Management API

Implement REST APIs for:

Create Ticket

A customer can create a ticket with:

- Customer name
- Customer email
- Subject
- Description
- Priority
- Category

Retrieve Ticket

Retrieve ticket details by ID.

List Tickets

Support:

- Pagination
- Filtering by:
 - Status
 - Priority

- Category

Update Ticket Status

Allow changing ticket status through predefined states.

Example statuses:

- OPEN
- IN_PROGRESS
- RESOLVED
- CLOSED

Python Engineering Expectations

The implementation should demonstrate:

- Proper schema validation
- Type hints
- Enums
- Clear project structure
- Separation of concerns
- Error handling
- Configuration management
- Logging
- Clean API responses
- Consistent coding standards

Example concepts expected:

- DTOs / request-response schemas
 - Service layer abstraction
 - Repository/data access separation
 - Environment-based configuration
 - Validation for invalid state transitions
-

Database Design

Use PostgreSQL and design a relational schema for:

- Tickets
- Ticket events/history
- Optional users/agents tables

Requirements:

- Proper normalization
- Constraints
- Indexes where appropriate
- Enum usage where appropriate
- Audit timestamps

Include:

- Database migrations
- Seed data for quick testing

[Bonus] Async Processing / Queue

When a new ticket is created:

- Publish a background job/event for processing.

Example background tasks:

- Generate an automatic ticket summary
- Assign ticket priority
- Simulate spam detection
- Simulate routing to the correct department

The processing result should be stored in the database.

You may use:

- AWS SQS
- Redis + Celery
- Redis + RQ
- Any equivalent lightweight queue solution

If queues are not implemented, provide a short design explanation describing how you would implement asynchronous processing.

API Design

Include:

- RESTful endpoint design
 - Proper HTTP status codes
 - Request validation
 - Error response format
 - OpenAPI / Swagger documentation
-

Dockerization

Provide a fully containerized setup using Docker Compose.

The project should run with a single command.

Expected services:

- API service
 - PostgreSQL
 - Optional worker service
 - Optional Redis
-

Documentation

Include a README with:

- Project overview
 - Architecture decisions
 - Setup instructions
 - Environment variables
 - How to run locally
 - API documentation access
 - Tradeoffs and assumptions
-

Deliverables

Provide a GitHub repository containing:

- Source code
- Docker configuration
- Database migrations
- Seed scripts
- README documentation

The application should be runnable locally with minimal setup.